

Small Step Semantics

@ClocksSugars (Not a computer scientist)

10/05/26

<https://softwarefoundations.cis.upenn.edu/lf-current/index.html>

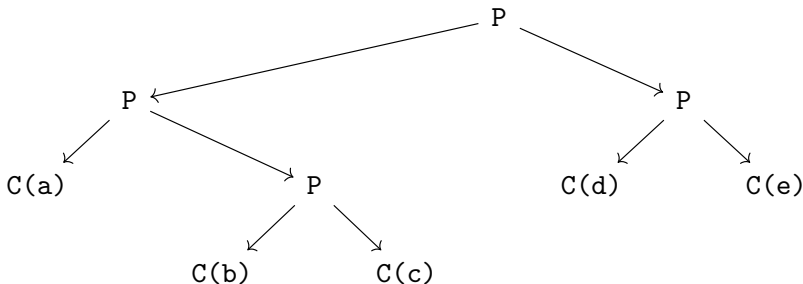
<https://softwarefoundations.cis.upenn.edu/plf-current/index.html>

An *Extremely* Simple Language

Introducing *Basic Addition* with **terms** P and C

```
1 Inductive tm : Type :=  
2   | C : nat -> tm      (* Constant *)  
3   | P : tm -> tm -> tm. (* Plus *)
```

$a + (b + c) + (d + e)$ is written (left associative) as...



Big Step vs Small Step

What kinds of propositions can we ask about a term in *Basic Addition*?

We can ask “does $(t : tm)$ evaluate to $(n : nat)$?”

```
1  Reserved Notation " t '==>' n " (at level 50, left associativity).
2
3  Inductive eval : tm -> nat -> Prop :=
4    | E_C : forall n,
5      C n ==> n
6    | E_P : forall t1 t2 n1 n2,
7      t1 ==> n1 ->
8      t2 ==> n2 ->
9      P t1 t2 ==> (n1 + n2)
10
11 where " t '==>' n " := (eval t n).
```

Big Step vs Small Step

We can speak of ‘small steps’ as individual steps a term of *Basic Addition* can take, and ask whether $(t1 : tm)$ steps into $(t2 : tm)$

```
1  Reserved Notation " t '-->' t' " (at level 40).
```

```
2  
3  Inductive step : tm -> tm -> Prop :=
```

```
4    | ST_PCC : forall n1 n2,  
5      P (C n1) (C n2) --> C (n1 + n2)
```

```
6    | ST_P1 : forall t1 t1' t2,  
7      t1 --> t1' ->
```

```
8      P t1 t2 --> P t1' t2
```

```
9    | ST_P2 : forall n1 t2 t2',  
10     t2 --> t2' ->
```

```
11     P (C n1) t2 --> P (C n1) t2'
```

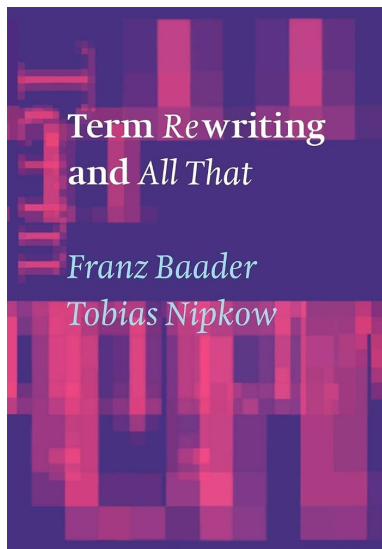
```
12  
13  where " t '-->' t' " := (step t t').
```

Desirable Properties for `step`

If we have defined a step-based-evaluation for a language, what could we *want* from it?

- Determinism: all terms proceed by only one step
- Values: a notion of what terms count as outputs of our program
- Normal forms: a notion of what terms cannot step
- Values are Normal Forms: proof that normal forms only cannot step when they are an output
- Progress: proof that all terms are either values or *will* step
- Confluence: (weakening of determinism) all sound terms eventually step to the same thing

Aside: Term Rewriting Theory



In the broader theory of *term rewriting*, what we've called a *step* is $R \subseteq \mathbf{tm} \times \mathbf{tm}$ such that if $(t1, t2) \in R$, we say $t1 \rightarrow t2$.

$\rightarrow$ also defines other relations:

$$x \xrightarrow{0} y$$

implies... $x = y$ (identity)

$$\xrightarrow{i+1} := \xrightarrow{i} \circ \rightarrow$$

or i -fold composition

$$\xrightarrow{+} := \bigcup_{i \geq 0} \xrightarrow{i}$$

(transitive closure)

$$\xrightarrow{*} := \xrightarrow{+} \cup \xrightarrow{0}$$

(reflexive transitive closure)

Aside: Broader Theory

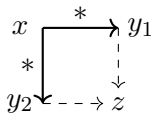
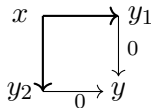
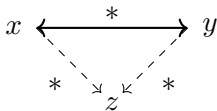
By a few other symbols

$$\longleftrightarrow^* := \rightarrow^* \cup \leftarrow^*$$

$$x \downarrow y := \exists z, x \xrightarrow{*} z \xleftarrow{*} y \text{ (joinable)}$$

we can express some famous properties

- Church-Rosser $x \longleftrightarrow^* y \implies x \downarrow y$
- Determinism $y_1 \leftarrow x \rightarrow y_2 \implies y_1 = y_2$
- Confluence $y_1 \xleftarrow{*} x \xrightarrow{*} y_2 \implies y_1 \downarrow y_2$



Aside: And Diagrams *are* Fun

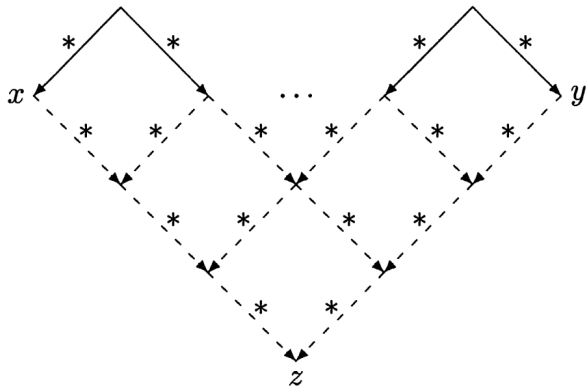


Fig. 2.3. Confluence implies the Church-Rosser property.

Multi-step

That's all to say we'll take $\xrightarrow{*}$ forward with us

```
1 Definition relation (X : Type) := X -> X -> Prop.
2
3 Inductive multi {X : Type} (R : relation X) : relation X :=
4   | multi_refl : forall (x : X), multi R x x
5   | multi_step : forall (x y z : X),
6                   R x y ->
7                   multi R y z ->
8                   multi R x z.
9
10 Notation " t '-->*' t' " := (multi step t t') (at level 40).
```

Some Common Tactics You'll Need: `intros`

Takes hypothesized items/propositions and brings them into context.

`intros x y1 y2 Hy1` will transform

```
(1/1) -----  
forall x y1 y2 : tm, x --> y1 → x --> y2 → y1 = y2
```

into

```
x, y1, y2 : tm  
Hy1 : x --> y1  
(1 / 1) -----  
x --> y2 → y1 = y2
```

Some Common Tactics You'll Need: `destruct`

Allows you to prove a proposition for each case of an inductive.

`destruct G as [G | G]` (where `as [G | G]` is optional explicit naming) will transform

```
t : tm
G : value t ∨ (exists t' : tm, t --> t')
(1 / 1) -----
value t
```

breaking up the sum-type (logical-or) $\vee$ into left or right cases

```
t : tm
G : value t
(1 / 2) -----
value t
```

```
t : tm
G : exists t' : tm, t --> t'
(2 / 2) -----
value t
```

Some Common Tactics You'll Need: `apply`

Applies a theorem/hypothesis to another hypothesis.

`apply` H transforms

```
H : P → Q → R
(1 / 1) -----
R
```

into

```
H : P → Q → R
(1 / 2) -----
P

(2 / 2) -----
Q
```

alternatively `apply` H1 `in` H2 `as` H3 on H1 : P → Q and
H2 : P can create a new hypothesis in context H3 : Q!

Some Common Tactics You'll Need: `induction`

Similar to `destruct` but will provide (usually with obligations) inductive hypotheses for the fields of each constructor. Pay attention to *exactly* what goal state you `induction` over!

```
x : tm
(1 / 1) -----
forall y1 y2 : tm, x --> y1 → x --> y2 → y1 = y2
```

by `induction` x as [| x1 IHx1 x2 IHx2] (`as` optional)

```
n : nat
(1 / 2) -----
forall y1 y2 : tm, C n --> y1 → C n --> y2 → y1 = y2
```

```
x1, x2 : tm
IHx1 : forall y1 y2 : tm, x1 --> y1 → x1 --> y2 → y1 = y2
IHx2 : forall y1 y2 : tm, x2 --> y1 → x2 --> y2 → y1 = y2
(2 / 2) -----
forall y1 y2 : tm, P x1 x2 --> y1 → P x1 x2 --> y2 → y1 = y2
```

Some Common Tactics You'll Need: `inversion`

Sometimes an inductive proposition only exists if its arguments satisfy certain pattern matches. `inversion` allows you to essentially ask rocq what had to be true for that proposition to exist and extract constraints; sometimes this behaves like `destruct`. So far as I know **lean does not have any direct analog to inversion**.

Let's say we had

```
1 Inductive Le : nat -> nat -> Set :=
2   | Le0 : forall n:nat, Le 0 n
3   | LeS : forall n m:nat, Le n m -> Le (S n) (S m).
```

then `inversion` H1 transforms

```
n, m : nat
H : Le (S n) m
(1/1) -----
SomeProp n m
```

Some Common Tactics You'll Need: `inversion`

```
n, m : nat
H : Le (S n) m
n0, m0 : nat
H1 : Le n m0
H0 : n0 = n
H2 : S m0 = m
(1/1) -----
P n (S m0)
```

It is common to follow up inversion with `inversion`; `subst` to compose the inversion tactic with the ‘substitute-all’ tactic, eliminating redundant equalities/hypotheses

```
n, m : nat
H : Le (S n) m
m0 : nat
H1 : Le n m0
H2 : S m0 = m
(1/1) -----
P n (S m0)
```

Other Common Tactics You'll Need

- `generalize dependent x` returns `x` from context to the goal state, useful if you want to change what you're inducting over
- `specialize (THEOREM ARGS) as NAME` can be used to add an existing theorem in a useful specific form, through judicious choice of arguments, to the context.
- When you have multiple goals, you can target them with bullet point markers (`-`, `+`, `*`), curly braces, or apply tactics by number i.e. `1,3,4: intros, all: inversion`.
- Since `inversion` also eliminates contradictions when there is no well formed inversion, B.C. Pierce's PLF book provides `solve_by_invert` and `solve_by_inverts N` as inversion based auto tactics. The keyword `try` will allow do-if-solve tactic application i.e.

```
all: try solve_by_inverts 3
```

Other Common Tactics You'll Need

- **unfold**: transform a defined term into its definition
- **exists t**: provide **t** as an example to an **exists** goal state
- **apply H1 in H2 as H3**: applies a contextual theorem (H1) to another one (H2) or provides it as a new hypothesis (H3) (without **as**, applies to H2, without **in** applies to goal state)

Some other not-quite-tactics which you'll find useful

Print step will print the definition of **step** (or anything else) that applies in a context.

Locate step will tell you where the definition of **step** (or anything else) that applies in your scope was defined

Search step will tell you where the **step** matching your scope (or anything else) also appears

What else you can do with this

Anything that is even slightly lambda calculus shaped!!

Examples:

See plf book chapters